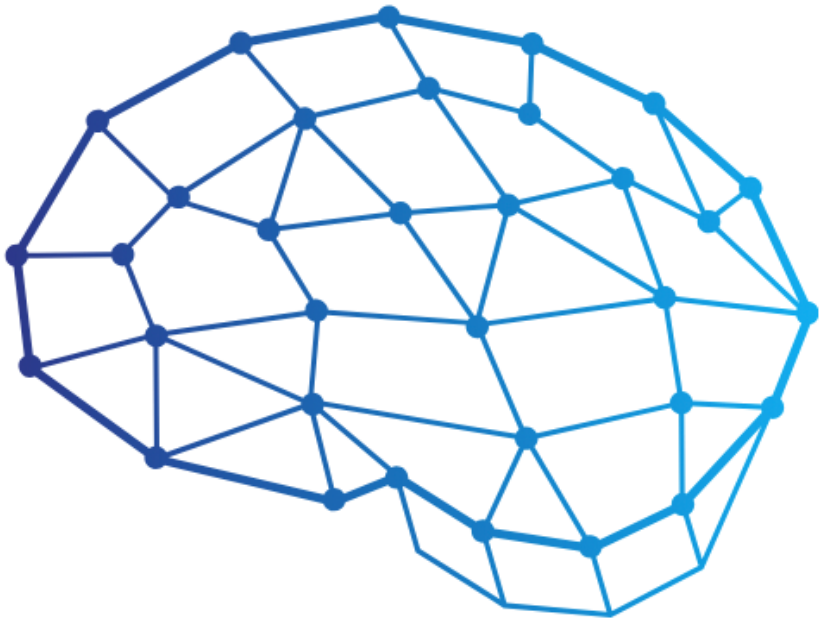
EMG Emotion Prediction

William Conley | Adrian Fudge | Oluwateniola "Teni" Adegbite
Dilan Mian | Alannis Davis

Introduction

ADSL



Goals

1. Create an emotion prediction model that can classify the emotional state for electromyography signals
2. Determine the effectiveness of electromyography data as a predictor of emotional state

Dataset: PeakAffectDS

PeakAffectDS

Greene, Nick[1] ; Livingstone, Steven R.[1]; Szymanski, Lech[1]

Show affiliations

Contributors

Supervisors: Livingstone, Steven[1]; Szymanski, Lech[1]

Show affiliations

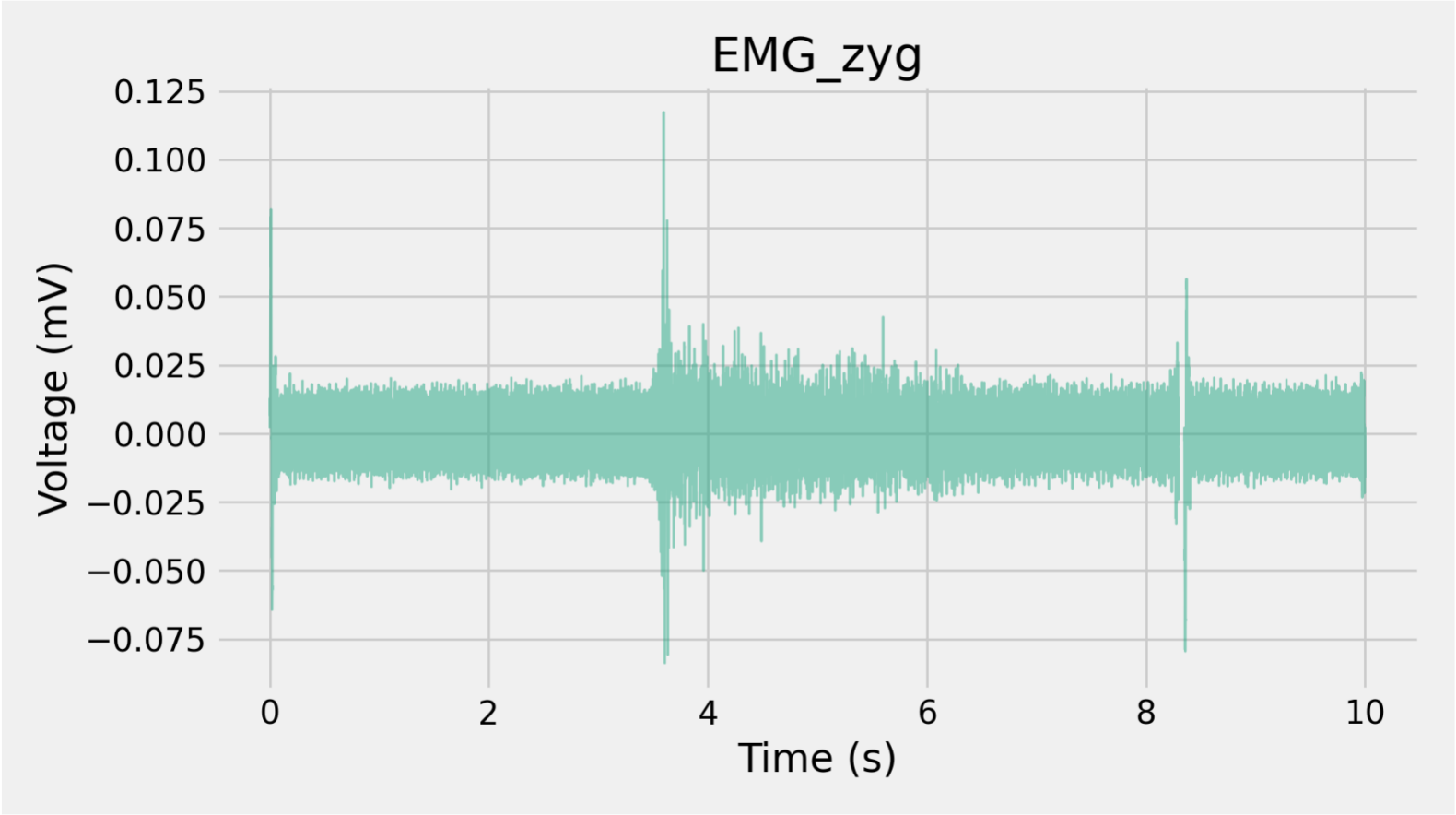
Contact Information

If you would like further information about PeakAffectDS, to purchase a commercial license, or if you experience any issues downloading files, please contact us at peakaffectds@gmail.com.

Description

PeakAffectDS contains 663 files (total size: 1.84 GB), consisting of 612 physiology files, and 51 perceptual rating files. The dataset contains 51 untrained research participants (39 female, 12 male), who had their body physiology recorded while watching movie clips validated to induce strong emotional reactions. Emotional conditions included: calm, happy, sad, angry, fearful, and disgust; along with baseline a neutral condition. Four physiology channels were recorded with a Biopac MP36 system: two facial muscles with fEMG (zygomaticus major, corrugator supercilii) using Ag/AgCl electrodes, heart activity with ECG using a 1-Lead, Lead II configuration, and respiration with a wearable strain-gauge belt. While viewing movie clips, participants indicated in real-time when they experienced a "peak" emotional event, including: chills, tears, or the startle reflex. After each clip, participants further rated their felt emotional state using a forced-choice categorical response measure, along with their felt Arousal and Valence. All data are provided in plaintext (.csv) format.

EMG Feature Extraction: EMGFlow



Emotion Prediction Model

```python
class EmotionPredictionModel(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = torch.nn.Linear(70, 256)
        self.fc2 = torch.nn.Linear(256, 128)
        self.fc3 = torch.nn.Linear(128, 64)
        self.fc4 = torch.nn.Linear(64, 6)

        self.dropout = torch.nn.Dropout(0.3)
        self.bn1 = torch.nn.BatchNorm1d(256)
        self.bn2 = torch.nn.BatchNorm1d(128)
        self.bn3 = torch.nn.BatchNorm1d(64)

        self.gelu = torch.nn.GELU()

    def forward(self, x):
        x = self.gelu(self.bn1(self.fc1(x)))
        x = self.dropout(x)
        x = self.gelu(self.bn2(self.fc2(x)))
        x = self.dropout(x)
        x = self.gelu(self.bn3(self.fc3(x)))
        x = self.fc4(x)
        return x
```

Development Decisions and Fine Tuning

```
Epoch: [196/200]        Loss: [1.21]    Accuracy: [48.00%]
Epoch: [197/200]        Loss: [1.18]    Accuracy: [50.91%]
Epoch: [198/200]        Loss: [1.19]    Accuracy: [44.00%]
Epoch: [199/200]        Loss: [1.17]    Accuracy: [47.27%]
Epoch: [200/200]        Loss: [1.23]    Accuracy: [45.82%]
Test Loss: [1.23]
Test Accuracy: [52.00%]
```

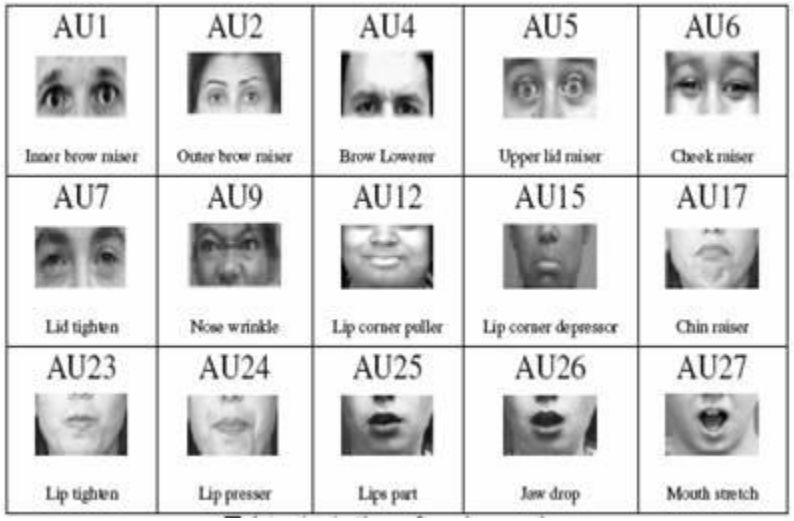
Results

```
Label 0: 100.00% accuracy
Label 1: 20.00% accuracy
Label 2: 0.00% accuracy
Label 3: 100.00% accuracy
Label 4: 0.00% accuracy
Label 5: 66.67% accuracy
```

predictions	0	1	2	3	5
labels					
0	4	0	0	0	0
1	1	1	0	3	0
2	1	0	0	2	0
3	0	0	0	5	0
4	2	0	2	1	0
5	1	0	0	0	2

0. Calm
1. Happy
2. Sad
3. Angry
4. Fearful
5. Disgust

Discussion


AU1
Inner brow raiser
AU2
Outer brow raiser
AU4
Brow Lowerer
AU5
Upper lid raiser
AU6
Cheek raiser
AU7
Lid tighten
AU9
Nose wrinkle
AU12
Lip corner puller
AU15
Lip corner depressor
AU17
Chin raiser
AU23
Lip tighten
AU24
Lip presser
AU25
Lips part
AU26
Jaw drop
AU27
Mouth stretch

Conclusion and Next Steps

GitHub: https://github.com/Willson/EMG-Emotion-Prediction

References

Chen, G., Chen, P., Shi, Y., Hsieh, C.-Y., Liao, B., & Zhang, S. (2019). Rethinking the usage of batch normalization and dropout in the training of deep neural networks. arXiv preprint arXiv:1905.05928. https://doi.org/10.48550/arXiv.1905.05928

Shah, V., & Youngblood, N. (2024). Leveraging continuously differentiable activation functions for learning in quantized noisy environments. arXiv preprint arXiv:2402.02593. https://doi.org/10.48550/arXiv.2402.02593

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research, 16*, 321–357. https://doi.org/10.1613/jair.953